



**¡Les damos la
bienvenida!**

¿Comenzamos?

Esta clase va a ser

- grabada
a

Clase 06. PROGRAMACIÓN BACKEND

Servidores Web

Temario

05

Administrador de paquetes NPM

- ✓ Node Js
- ✓ Módulos nativos
- ✓ NPM
- ✓ Instalaciones globales y locales

06

Servidores web

- ✓ [Protocolo HTTP](#)
- ✓ [Servidor con express.js](#)
- ✓ [Objeto request](#)

07

Express avanzado

- ✓ Status de protocolo http
- ✓ Concepto de API REST
- ✓ Métodos POST, PUT, DELETE

Objetivos de la clase

- **Entender** el protocolo HTTP
- **Aprender** a crear un servidor utilizando el módulo nativo http
- **Crear** un servidor de express
- **Profundizar** sobre las peticiones GET de un servidor express.

Glosario

Nodejs: Entorno de ejecución de javascript.
Es el corazón de nuestros programas.

Módulo: Conjunto de funciones y objetos que permiten resolver un problema específico.

Módulo nativo: Módulos que vienen incluidos al instalar node js, no requieren instalación, sólo importación.

package.json: Archivo de especificación del proyecto, sirve para contener las dependencias a utilizar en el proyecto.

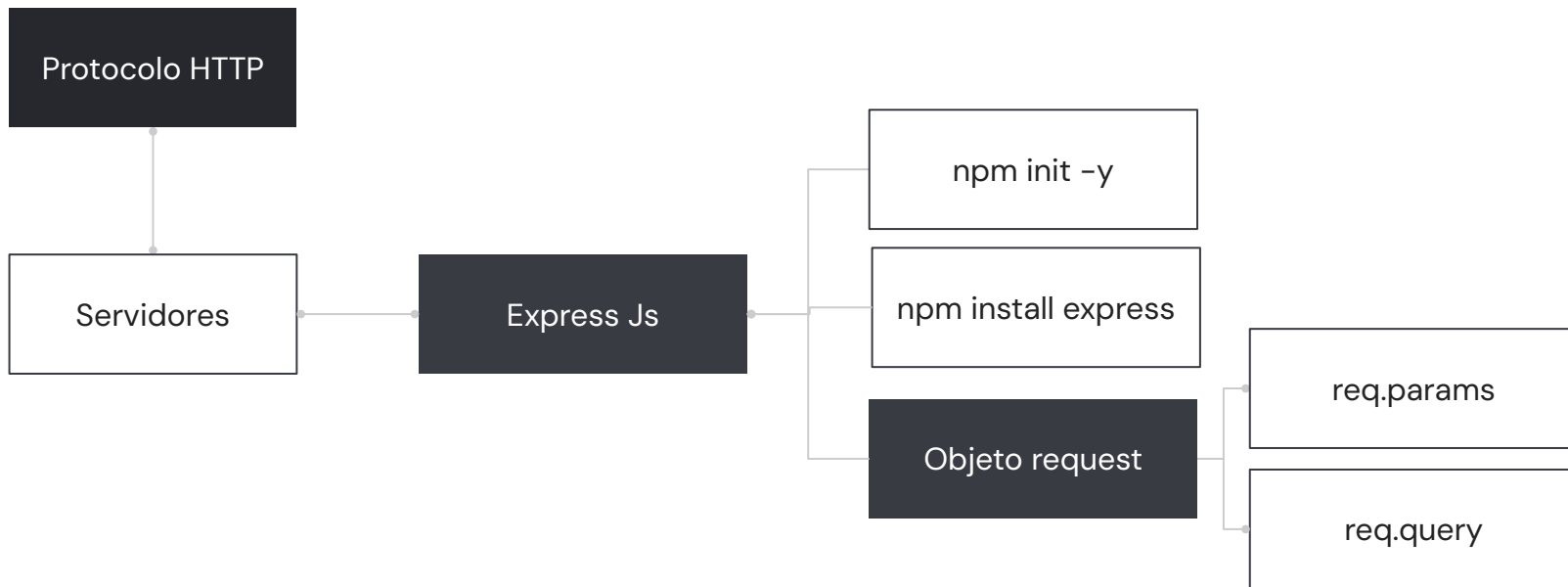
Dependencias: Todos los módulos externos a nuestro proyecto que necesitamos instalar para poder correr el proyecto.

Dependencia local: Dependencias que sólo viven dentro de nuestro proyecto.

Dependencia global: Dependencias que se instalan a nivel computador y pueden ser utilizadas en cualquier proyecto.

Versión: Cada una de las etapas y cambios que representa nuestro proyecto, pueden ser mayores, menores o parches.

MAPA DE CONCEPTOS



Entendiendo el protocolo HTTP

HTTP

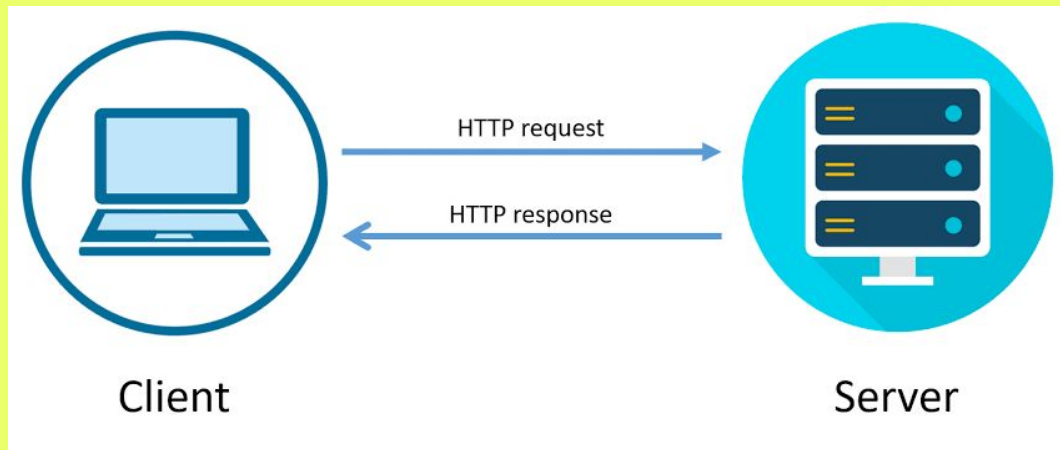
HTTP (Hyper Text Transfer Protocol) refiere a un **protocolo**, el cual es un conjunto de reglas que permite la comunicación entre dos o más sistemas. Gracias a este protocolo, las computadoras saben comunicarse entre sí y permiten comunicarse con servidores para obtención de datos. (lo podemos ver en todas las páginas que visitamos)



HTTP

¿Cómo funciona?

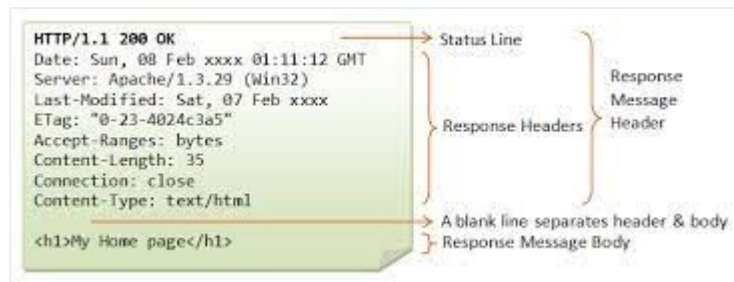
Se basa en un modelo de **petición - respuesta**. De manera que el **cliente** tiene que hacer una petición de información, entonces **el servidor** se encarga de responder con dicha información.



¿Qué tipo de información pedimos?

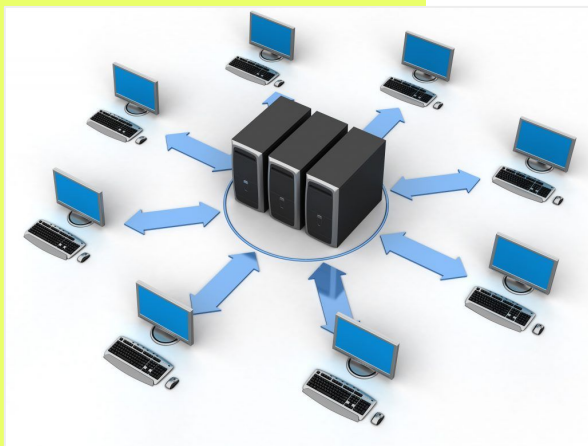
Al hacer una petición, estamos solicitando al servidor ciertos recursos, estos pueden ser:

- ✓ Un dato como un nombre, una fecha, una edad...
- ✓ información más compleja como una imagen, un vídeo.
- ✓ Un archivo para descargar
- ✓ ¡Incluso una página web completa!



Ejemplo de la respuesta de un servidor, devolviendo una etiqueta h1

Maneja múltiples peticiones



Cuando programamos nuestro servidor, lo hacemos para escuchar peticiones, la pregunta es: **¿de quién escuchar las peticiones?**

¡Bajo su **configuración por defecto**, un servidor puede escuchar múltiples peticiones de múltiples clientes al mismo tiempo!



¿Y si el servidor se apaga?

Sabemos que un programa se inicia, hace sus operaciones, y luego finaliza.

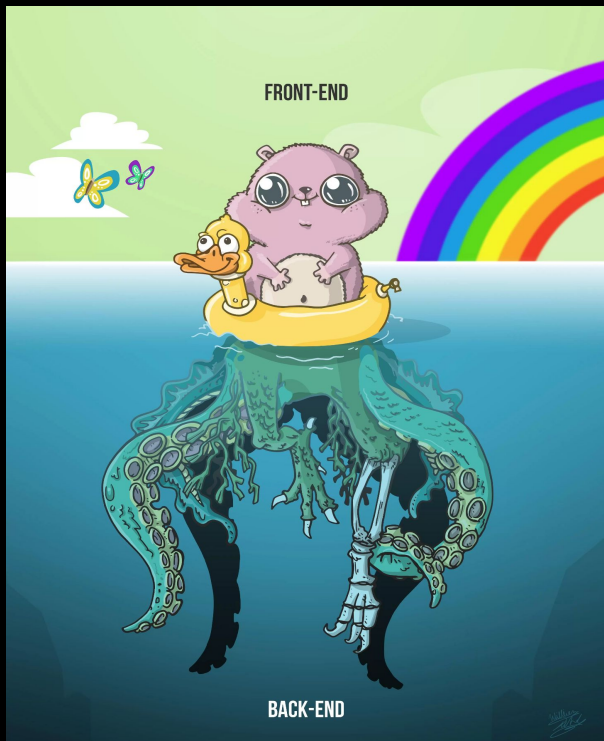
¿Qué tiene de diferente el servidor? ¿Cómo puede recibir peticiones en diferentes períodos de tiempo sin finalizar? 🤔

¡Importante!

El cliente siempre es quien hace las peticiones (**requests**) y el servidor siempre será quien hace las respuestas (**responses**) . Cuando hacemos frontend, somos clientes. ¡En este curso te tocará ser el servidor y devolver respuestas!



PARA RECORDAR



Mientras que el **FrontEnd** se centra en la experiencia del usuario, **¡que esta sea lo más presentable y amigable posible!**

...el **BackEnd** tiene una estructura interna para que todo funcione correctamente. **¡Que no lo vea el usuario!**

**¿Qué necesitamos
antes de comenzar?**

Instalar nodemon de manera global



Como nuestro servidor se mantiene escuchando todo el tiempo, los cambios que hagamos en el código no se reflejan automáticamente. Por ello **tenemos que apagar el servidor y levantarlo nuevamente**.

Sólo así podemos visualizar los cambios que hagamos en el código (así sea un simple `console.log()`)

Nodemon nos permitirá **reiniciar automáticamente** el servidor en cuanto detecta que hay cambios en el código. De esta manera, podemos concentrarnos en el código, sin tener que realizar el reinicio manual cada vez que queremos ver algo.

Para instalarlo, basta correr el comando:

```
npm install -g nodemon
```



Ejemplo en vivo

- ✓ Crear un servidor con el módulo nativo de nodejs "http". Setear una respuesta que contenga el mensaje "¡Mi primer hola mundo desde backend!"
- ✓ El servidor debe escuchar en el puerto 8080. (Correr con nodemon)
- ✓ Probar el servidor desde el navegador.
- ✓ Hacer algún cambio en código y corroborar que se reinicie automáticamente.

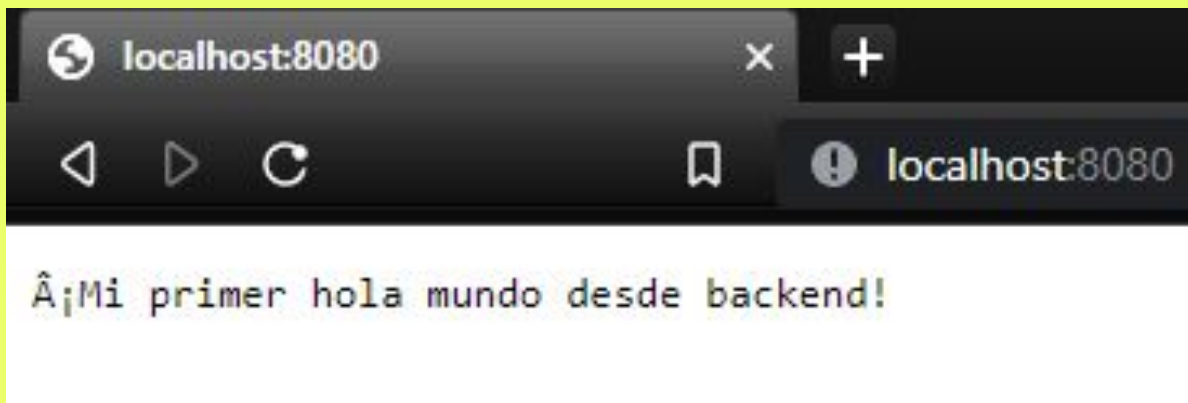
Parte 1: Realizando el servidor

```
const http = require('http');

const server = http.createServer((request, response) => {
  response.end("¡Mi primer hola mundo desde backend!");
});

server.listen(8080, () => {
  console.log("Listening on port 8080")
});
```

Parte 2: Probando desde navegador



Servidor con Express js

¿Qué es Express js?

Express js es un framework minimalista que permitirá desarrollar servidores más complejos.

Éste nos facilitará:

- ✓ Utilizar diferentes rutas para las peticiones.
- ✓ Mejorar la estructura de nuestro proyecto.
- ✓ Manejar funcionalidades más complejas y utilización de middlewares.

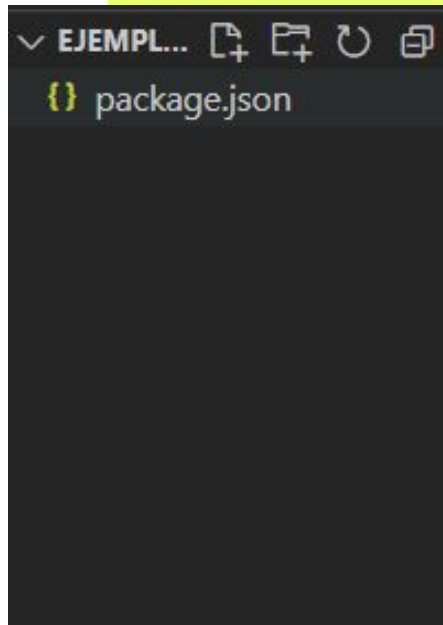


Preparación de un proyecto de Express js

Paso 1: npm init -y

Express no es nativo de nodejs, por lo tanto, necesitaremos primero contar con un **package.json** para gestionar las dependencias a instalar.

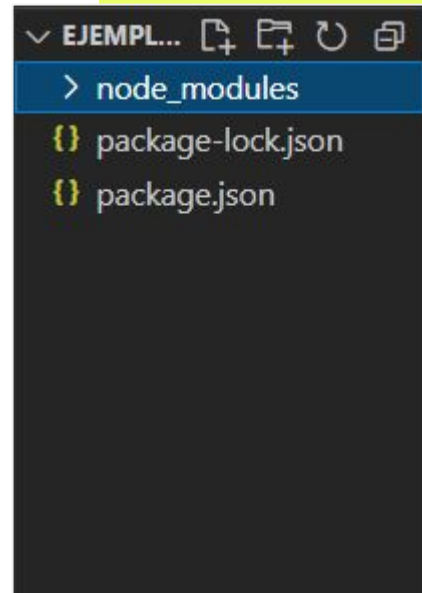
Una vez que tenemos package.json en nuestra carpeta, podemos continuar instalando dependencias.



Paso 2: npm install express

Procedemos a instalar de manera local express js. Al ejecutar este comando, notaremos cómo se genera una carpeta node_modules, que es donde se encuentra almacenado express

A partir de este punto, ya contamos con la estructura elemental instalada, el resto es más “flexible”.



Paso 3: Estructurar el proyecto

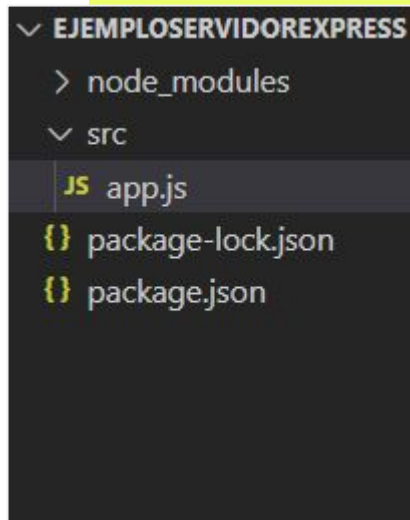
Se recomienda tener una carpeta src, donde vivirá todo nuestro código, dentro del cual crearemos un archivo con el nombre "app.js"

Finalmente, el archivo app.js ya puede importar la dependencia instalada de **express js**, ya sea por commonjs:

```
const express = require('express');
```

o bien por module (recordar colocar el type:"module" en package.json):

```
import express from 'express';
```

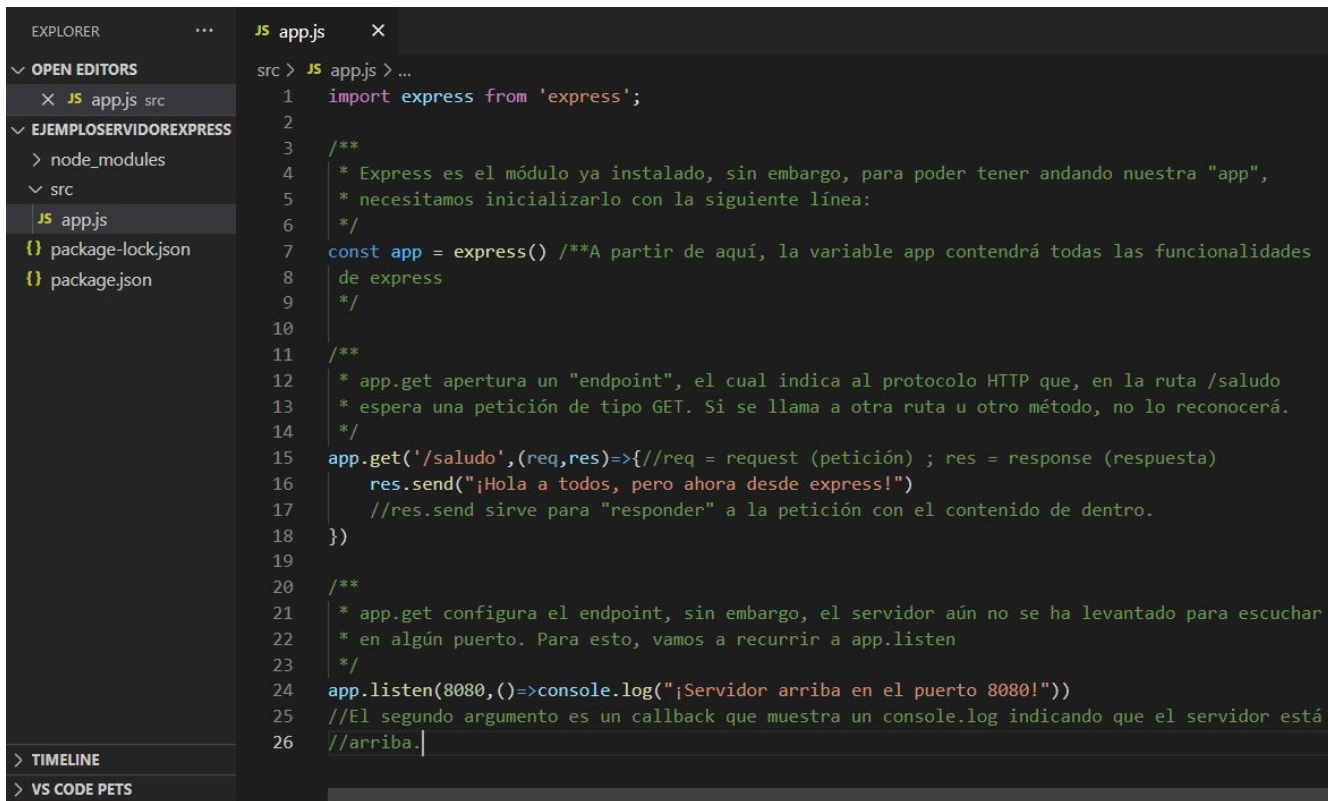




Ejemplo de una consulta en Express

- ✓ Estructurar un servidor basado en express, el cual escuche peticiones en el puerto 8080
- ✓ Realizar una función para el método GET en la ruta '/saludo', el cual responderá con "¡Hola a todos, pero ahora desde express!"
- ✓ Ejecutar con nodemon y probar en el navegador el endpoint generado

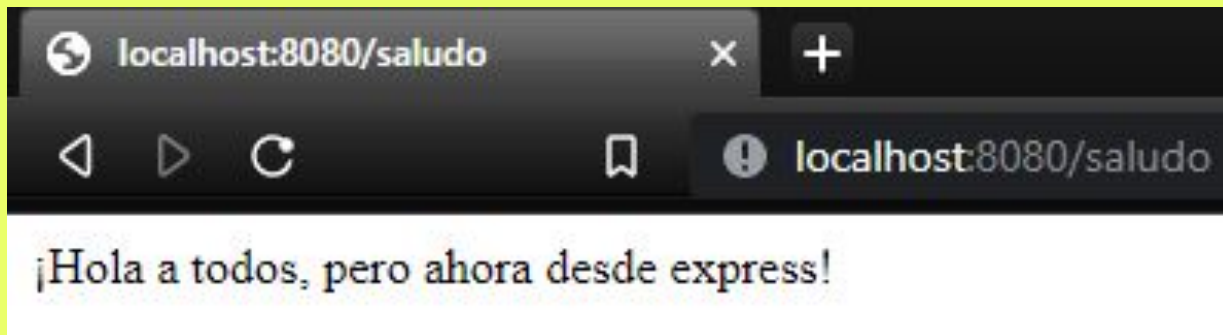
Parte 1: implementando el proyecto



The image shows a VS Code editor window with a dark theme. On the left, the Explorer sidebar shows a file tree with 'src' expanded, containing 'app.js', 'package-lock.json', and 'package.json'. The main editor area shows the content of 'app.js' with line numbers 1 through 26. The code imports 'express' and sets up a basic HTTP server with a GET endpoint at '/saludo' that responds with '¡Hola a todos, pero ahora desde express!'. The server listens on port 8080.

```
src > JS app.js > ...
1  import express from 'express';
2
3  /**
4   * Express es el módulo ya instalado, sin embargo, para poder tener andando nuestra "app",
5   * necesitamos inicializarlo con la siguiente línea:
6   */
7  const app = express() /**A partir de aquí, la variable app contendrá todas las funcionalidades
8   de express
9   */
10
11 /**
12  * app.get apertura un "endpoint", el cual indica al protocolo HTTP que, en la ruta /saludo
13  * espera una petición de tipo GET. Si se llama a otra ruta u otro método, no lo reconocerá.
14  */
15  app.get('/saludo',(req,res)=>{/req = request (petición) ; res = response (respuesta)
16    res.send("¡Hola a todos, pero ahora desde express!")
17    //res.send sirve para "responder" a la petición con el contenido de dentro.
18  })
19
20 /**
21  * app.get configura el endpoint, sin embargo, el servidor aún no se ha levantado para escuchar
22  * en algún puerto. Para esto, vamos a recurrir a app.listen
23  */
24  app.listen(8080,()=>console.log("¡Servidor arriba en el puerto 8080!"))
25  //El segundo argumento es un callback que muestra un console.log indicando que el servidor está
26  //arriba.
```

Parte 2: Probando en el navegador en endpoint especificado





Otras respuestas con express

Conociendo otros tipos de elementos que podemos responder con express.

Duración: 10 min



ACTIVIDAD EN CLASE

Otras respuestas con express

Crear un proyecto basado en express js, el cual cuente con un servidor que escuche en el puerto 8080. Además de configurar los siguientes endpoints:

- ✓ El endpoint del método GET a la ruta `/bienvenida` deberá devolver un html con letras en color azul, en una string, dando la bienvenida.
- ✓ El endpoint del método GET a la ruta `/usuario` deberá devolver un objeto con los datos de un usuario falso: `{nombre, apellido, edad, correo}`





Break

¡10 minutos y volvemos!

Objeto request

Objeto request

Al hacer los endpoints, hasta el momento sólo hemos utilizado el elemento “res”, el cual utilizamos para responder a una petición.

Sin embargo, estos últimos ejercicios hemos dejado de lado el “req” que viene a su izquierda, ahora abordaremos qué utilidad tiene y cómo podemos utilizarlo.

El objeto req cuenta con tres propiedades principales: **req.query**, **req.params**, **req.body**.

El día de hoy abordaremos req.query y req.params, en la próxima clase profundizaremos sobre req.body

req.params

req.params

Se utiliza cuando necesitamos obtener elementos dinámicos desde la ruta que está llamando el cliente.

para poder definir un “parámetro” dentro de la ruta a trabajar, basta con colocar el símbolo de dos puntos : antes del parámetro, de esta manera, express reconoce que queremos que ese elemento sea dinámico.

Definiendo endpoints con params

JS ejemploParams.js X

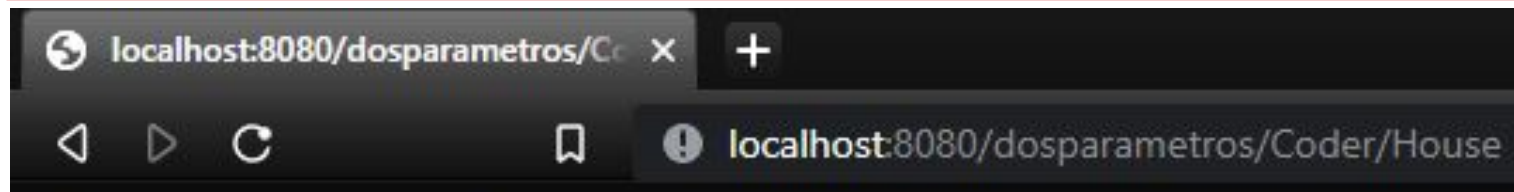
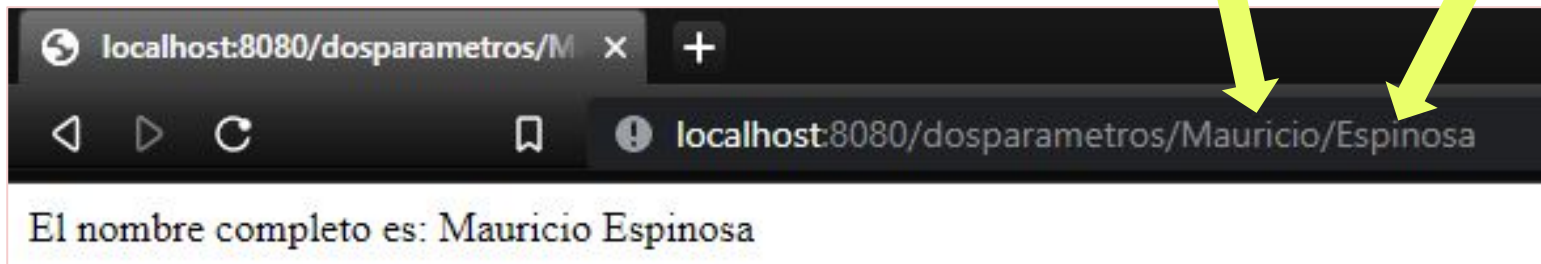
src > JS ejemploParams.js > ...

```
1  import express from 'express';
2
3  const app = express();
4  /**
5   * Utilizamos los dos puntos ( : ) para indicar que queremos que ese sea el parámetro.
6   * Esto nos permite crear una ruta dinámica que pueda recibir cualquier parámetro, en lugar
7   * de tratar de adivinarlo. Ahora podemos ingresar cualquier nombre desde la url, en lugar
8   * de tener que registrar 100000000000 rutas con 100000000000 nombres diferentes
9   */
10 app.get('/unparametro/:nombre',(req,res)=>{
11     //:parametro ahora se encontrará dentro del objeto req.params
12     console.log(req.params.nombre)
13     res.send(`¡Bienvenid@, ${req.params.nombre}!`)
14 })
15 app.get('/dosparametros/:nombre/:apellido',(req,res)=>{
16     //:nombre y :apellido ahora se encontrarán dentro del objeto req.params
17     //¡Podemos definir en nuestro endpoint cuantos parámetros necesitemos!
18     res.send(`El nombre completo es: ${req.params.nombre} ${req.params.apellido}`)
19 })
20
21 app.listen(8080,()=>console.log("¡Listo para recibir peticiones!"))
```

Probando endpoint de dos parámetros,
pasando nombres de manera dinámica

:nombre

:apellido





Ejemplo en vivo: Caso práctico de uso de params

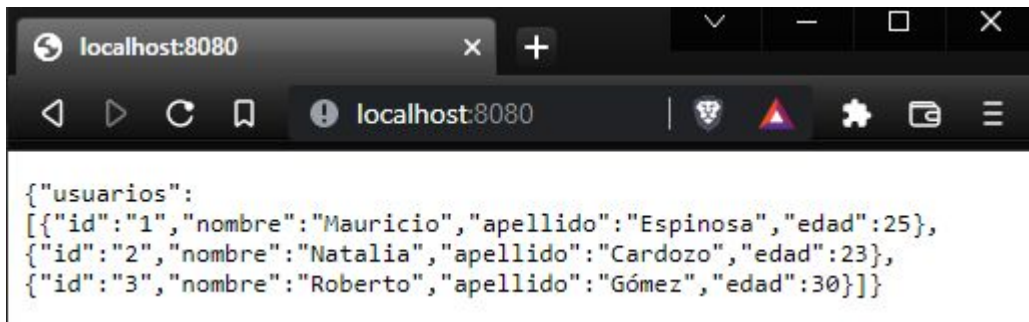
- ✓ Dado un arreglo de objetos de tipo usuario, realizar un servidor en express que permita obtener dichos usuarios.
- ✓ La ruta raíz '/' debe devolver todos los usuarios
- ✓ la ruta /:userId debe devolver sólo al usuario con dicho Id.

Ejemplo: Implementación

JS casoPrácticoParams.js X

```
src > JS casoPrácticoParams.js > [0] usuarios
1   import express from 'express';
2
3   const app = express();
4   const usuarios = [
5     {id:"1",nombre:"Mauricio",apellido:"Espinosa",edad:25},
6     {id:"2",nombre:"Natalia",apellido:"Cardozo",edad:23},
7     {id:"3",nombre:"Roberto",apellido:"Gómez",edad:30}
8   ]
9
10  app.get('/',(req,res)=>{
11    res.send({usuarios}) //Se recomienda enormemente mandar los datos en formato objeto en lugar
12    //de enviarlos como un array solo, esto permite que, si vamos a meter más info en el futuro
13    //no tengamos que cambiar el tipo de respuesta de lado del cliente.
14  })
15  app.get('/:idUserio',(req,res)=>{
16    let idUsuario = req.params.idUsuario; //Obtenemos el id del usuario a trabajar
17    /**
18     * Procedemos a buscar al usuario que tenga el id pasado por params.
19     */
20    let usuario = usuarios.find(u=>u.id===idUserio);
21    //Si no encuentra al usuario, debe finalizar la función devolviendo un error
22    if(!usuario) return res.send({error:"Usuario no encontrado"})
23
24    //En caso de que no haya finalizado la función, significa que el usuario sí se encontró.
25    res.send({usuario})
26  })
27  app.listen(8080,()=>console.log("Listo para probar caso práctico"))
```

Prueba ruta raíz: Debe devolver todos los usuarios.



A screenshot of a web browser window with the address bar showing 'localhost:8080'. The browser displays a JSON response from the root endpoint. The response is a JSON object with a key 'usuarios' containing an array of three user objects. Each user object has keys for 'id', 'nombre', 'apellido', and 'edad'.

```
{
  "usuarios": [
    {
      "id": "1",
      "nombre": "Mauricio",
      "apellido": "Espinosa",
      "edad": 25
    },
    {
      "id": "2",
      "nombre": "Natalia",
      "apellido": "Cardozo",
      "edad": 23
    },
    {
      "id": "3",
      "nombre": "Roberto",
      "apellido": "Gómez",
      "edad": 30
    }
  ]
}
```

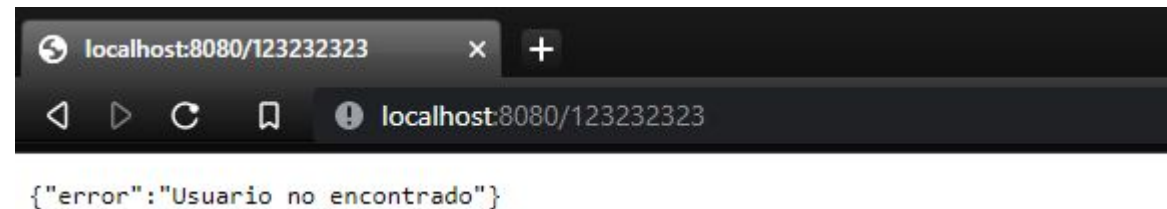
Prueba con parámetro válido: Debe encontrar al usuario y devolver sólo ese usuario.



A screenshot of a web browser window with the address bar showing 'localhost:8080/1'. The browser displays a JSON response for the endpoint with a valid ID. The response is a JSON object with a key 'usuario' containing a single user object.

```
{
  "usuario": {
    "id": "1",
    "nombre": "Mauricio",
    "apellido": "Espinosa",
    "edad": 25
  }
}
```

Prueba con parámetro inválido: Debe arrojar error al no encontrar usuario con ese id.



A screenshot of a web browser window with the address bar showing 'localhost:8080/123232323'. The browser displays a JSON response for the endpoint with an invalid ID. The response is a JSON object with a key 'error' containing the message 'Usuario no encontrado'.

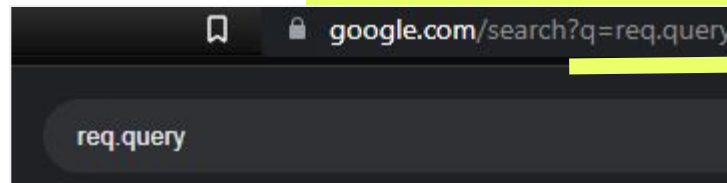
```
{
  "error": "Usuario no encontrado"
}
```

req.query

req.query

Como su nombre lo indica, query refiere a las múltiples consultas que se pueden hacer a un determinado endpoint, basta con que en la url coloquemos el símbolo **?**, entonces express reconocerá que hay que meter información al objeto **req.query** para poder utilizarlo en el endpoint.

Cuando buscamos algo en nuestro navegador, llamamos a un endpoint haciendo un determinado query.



¡Importante!

Conforme incrementa el dinamismo en las urls, es importante configurar el servidor para que reciba datos complejos desde la url, por ello hay que utilizar la línea:

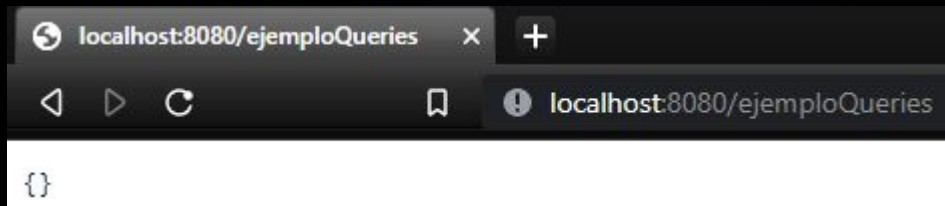
```
app.use(express.urlencoded({extended:true}))
```

La línea anterior permitirá que el servidor pueda interpretar mejor los datos complejos que viajen desde la url, y mapearlos correctamente en el req.query

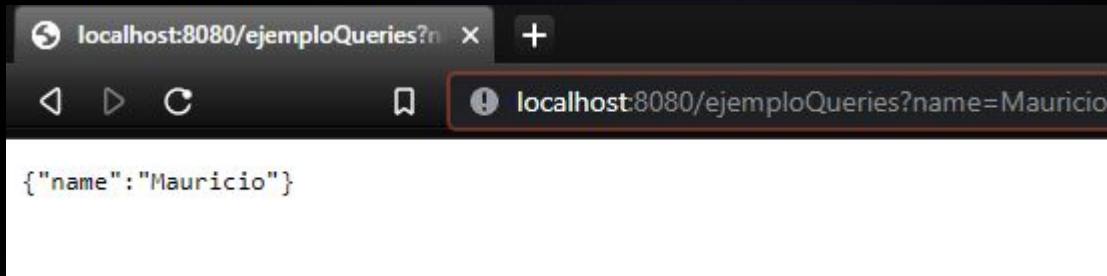
Definición de estructura de un endpoint con req.query

```
JS ejemploQuery.js X
src > JS ejemploQuery.js > ...
1  import express from 'express';
2
3  const app = express();|
4
5  app.use(express.urlencoded({extended:true}))
6
7  app.get('/ejemploQueries',(req,res)=>{
8    /**
9     * Nota algo interesante: Aquí no es necesario adelantarnos al parámetro que tiene que
10     * meter el cliente. Con el simple hecho de llamar al endpoint, el cliente puede
11     * meter los queries que necesite desde la url con el símbolo ?
12     */
13     let consultas = req.query;
14     /**
15     * A diferencia de req.params, aquí no tengo contemplado qué tipo de cosas me pueden pedir
16     * aunque sí podemos delimitarlo haciendo un destructuring
17     */
18     let {nombre,apellido,edad} = req.query;
19     /**
20     * Así, no importa lo que llegue del query, sólo extraeré el nombre,el apellido y la edad
21     * Esto aumenta la seguridad del servidor porque evitamos recibir elementos extraños desde
22     * la url
23     */
24     res.send(consultas)
25  })
26
27  app.listen(8080,()=>console.log("Listo para probar queries"))
```

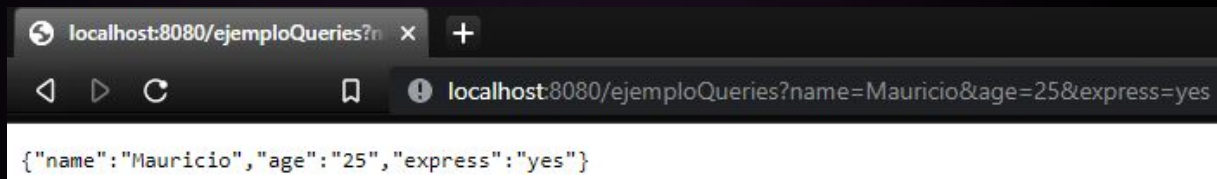
Si llamamos el endpoint sin query, notamos que el objeto req.query viene vacío.



Si llamamos el mismo endpoint, ahora con el símbolo ? para definir un query param, notamos cómo ahora aparece en el objeto.



Finalmente, podemos poner tantos queries queramos utilizando el símbolo & Y notamos cómo ahora el objeto req.query cuenta con más de un query param para poder ser utilizado.





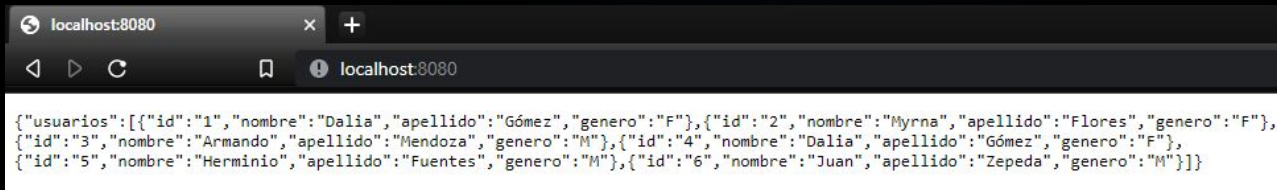
Ejemplo en vivo: Caso práctico de uso de req.query

- ✓ Dado un arreglo de objetos de tipo usuario, vamos a hacer un filtro por género
- ✓ La ruta raíz '/' debe devolver todos los usuarios, pero ahora colocaremos un query param con ?, indicando que queremos un género específico. en caso de enviarlo sin query, debe devolver a todos los usuarios.

Ejemplo:
implementando
endpoint que
utiliza
req.query

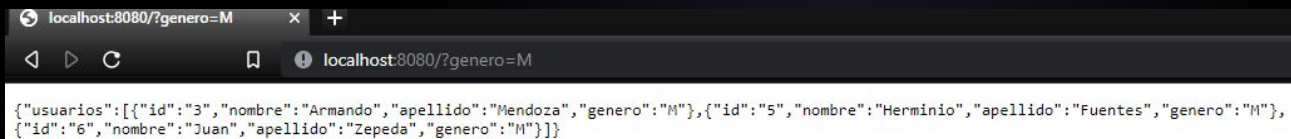
```
JS casoPrácticoQuery.js X
src > JS casoPrácticoQuery.js > app.get('/') callback
1  import express from 'express';
2
3  const app = express();
4
5  app.use(express.urlencoded({extended:true}))
6
7  //Filtraremos por género masculino (M) y femenino (F)
8  const usuarios = [
9    {id:"1",nombre:"Dalia",apellido:"Gómez",genero:"F"},
10   {id:"2",nombre:"Myrna",apellido:"Flores",genero:"F"},
11   {id:"3",nombre:"Armando",apellido:"Mendoza",genero:"M"},
12   {id:"4",nombre:"Dalia",apellido:"Gómez",genero:"F"},
13   {id:"5",nombre:"Herminio",apellido:"Fuentes",genero:"M"},
14   {id:"6",nombre:"Juan",apellido:"Zepeda",genero:"M"},
15 ]
16
17
18 app.get('/',(req,res)=>{
19   let genero = req.query.genero;
20   //Si no se ingresó género, o el género no es M ni es F, no vale el filtro.
21   if(!genero||(genero!="M"&&genero!="F")) return res.send({usuarios})
22   //En caso contrario, continuamos con el proceso de filtro.
23   let usuariosFiltrados = usuarios.filter(usuario=>usuario.genero===genero);
24   res.send({usuarios:usuariosFiltrados})
25 })
26
27 app.listen(8080,()=>console.log("Preparado para hacer filtros"))
```

Llamar al endpoint sin query permite que lleguen todos los usuarios sin filtrar



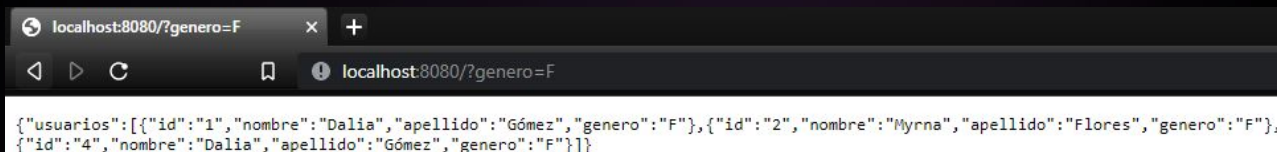
```
{
  "usuarios": [
    {
      "id": "1",
      "nombre": "Dalia",
      "apellido": "Gómez",
      "genero": "F"
    },
    {
      "id": "2",
      "nombre": "Myrna",
      "apellido": "Flores",
      "genero": "F"
    },
    {
      "id": "3",
      "nombre": "Armando",
      "apellido": "Mendoza",
      "genero": "M"
    },
    {
      "id": "4",
      "nombre": "Dalia",
      "apellido": "Gómez",
      "genero": "F"
    },
    {
      "id": "5",
      "nombre": "Herminio",
      "apellido": "Fuentes",
      "genero": "M"
    },
    {
      "id": "6",
      "nombre": "Juan",
      "apellido": "Zepeda",
      "genero": "M"
    }
  ]
}
```

Enviando el query con genero = M, notamos cómo devuelve sólo los usuarios con género M.



```
{
  "usuarios": [
    {
      "id": "3",
      "nombre": "Armando",
      "apellido": "Mendoza",
      "genero": "M"
    },
    {
      "id": "5",
      "nombre": "Herminio",
      "apellido": "Fuentes",
      "genero": "M"
    },
    {
      "id": "6",
      "nombre": "Juan",
      "apellido": "Zepeda",
      "genero": "M"
    }
  ]
}
```

Mismo caso para el género F.



```
{
  "usuarios": [
    {
      "id": "1",
      "nombre": "Dalia",
      "apellido": "Gómez",
      "genero": "F"
    },
    {
      "id": "2",
      "nombre": "Myrna",
      "apellido": "Flores",
      "genero": "F"
    },
    {
      "id": "4",
      "nombre": "Dalia",
      "apellido": "Gómez",
      "genero": "F"
    }
  ]
}
```

¡Acabamos de hacer un filtro de usuarios perfectamente funcional!

¿Qué diferencia hay con params?

La principal diferencia que hay entre req.params y req.query, es que en req.query puedo meter la cantidad de consultas que yo así desee, ya que las queries no vienen inmersas en la ruta, sino que son un elemento aparte.

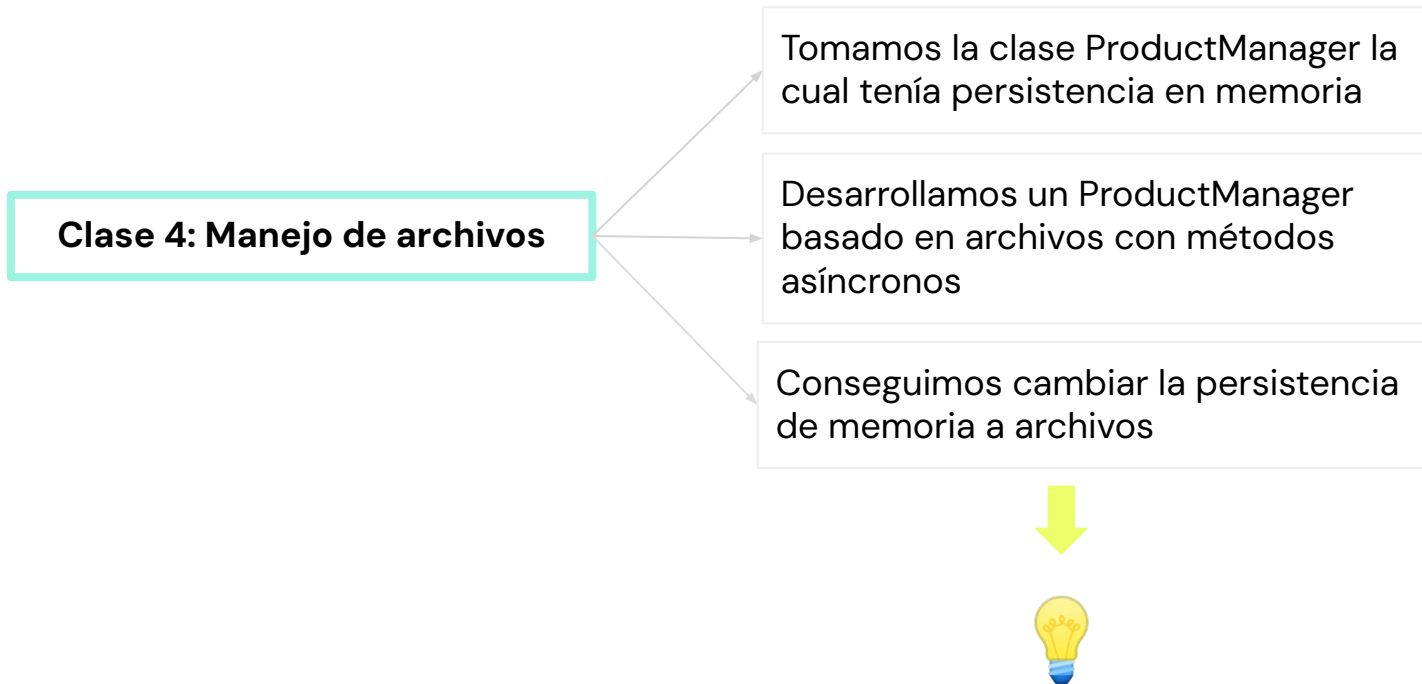
Así, si desconozco el número de cosas que se van a consultar en mi ruta, la mejor opción es utilizar **queries**, mientras que, si sólo necesito un número específico y reducido de parámetros, habría que optar por **params**

Al final, no hay una mejor que otra, sirven para casos diferentes e incluso podemos utilizar ambas en la misma consulta.



Servidor con express

Recordemos





Servidor con express

Consigna

- ✓ Desarrollar un servidor basado en express donde podamos hacer consultas a nuestro archivo de productos.

Aspectos a incluir

- ✓ Se deberá utilizar la clase ProductManager que actualmente utilizamos con persistencia de archivos.
- ✓ Desarrollar un servidor express que, en su archivo app.js importe al archivo de ProductManager que actualmente tenemos.

Aspectos a incluir

- ✓ El servidor debe contar con los siguientes endpoints:
 - ruta '/products', la cual debe leer el archivo de productos y devolverlos dentro de un objeto. Agregar el soporte para recibir por query param el valor ?limit= el cual recibirá un límite de resultados.
- Si no se recibe query de límite, se devolverán todos los productos
- Si se recibe un límite, sólo devolver el número de productos solicitados



Servidor con express

- ruta '/products/:pid', la cual debe recibir por req.params el pid (product Id), y devolver sólo el producto solicitado, en lugar de todos los productos.

Sugerencias

- Tu clase lee archivos con promesas. recuerda usar async/await en tus endpoints
- Utiliza un archivo que ya tenga productos, pues el desafío sólo es para gets.

Formato del entregable

- ✓ Link al repositorio de Github con el proyecto completo, el cual debe incluir:
 - carpeta src con app.js dentro y tu ProductManager dentro.
 - package.json con la info del proyecto.
 - NO INCLUIR LOS node_modules generados.

[Testing de este entregable](#)

¿Preguntas?

Opina y valora
esta clase

Resumen de la clase hoy

- ✓ Repaso sobre Node js
- ✓ Manejo de módulos nativos de Nodejs
- ✓ package.json y NPM
- ✓ Instalaciones globales y locales
- ✓ Versionado

Muchas gracias.

#DemocratizandoLaEducación